Tutorial PWA de SolidarianID

De cero a PWA instalable con cache, service worker y web push

SolidarianID

26 de abril de 2026

Indice

1 Objetivo del documento **3**

2 PWA desde cero **3**

3 Resumen de la rama `next-pwa` **4**
3.1 Fase 1: base instalable . . . 4
3.2 Fase 2: service worker y cache . . . 4
3.3 Fase 3: permisos y boton de notificaciones . . . 5
3.4 Fase 4: backend de push subscriptions . . . 5
3.5 Fase 5: envio de webpush y estrategias . . . 5

4 Frontend: hacer la app instalable **6**
4.1 Manifest . . . 6
4.2 Metadatos en layout . . . 7

5 Frontend: Serwist y service worker **7**
5.1 Por que Serwist . . . 7
5.2 Provider de Serwist . . . 8
5.3 Ruta que sirve el service worker . . . 8

6 Estrategia de cache **8**
6.1 Regla 1: no cachear metodos que no sean GET . . . 8
6.2 Regla 2: no cachear rutas sensibles . . . 9
6.3 Regla 3: usar cache por defecto para assets seguros . . . 9
6.4 Regla 4: limpiar caches antiguas . . . 9
6.5 Regla 5: navigation preload . . . 10

7 Frontend: activacion de web push **10**
7.1 Comprobar soporte . . . 10
7.2 Pedir permiso . . . 10
7.3 Crear push subscription . . . 10
7.4 Enviar la subscription al backend . . . 11
7.5 Desactivar notificaciones . . . 11

8 Frontend: boton de preferencias **11**

9 Backend: modelo de subscription **12**
9.1 Metodos de dominio . . . 13

10 Backend: persistencia **13**

11 Backend: consultas importantes **14**
11.1 Buscar por usuario . . . 14
11.2 Buscar por comunidad . . . 14
11.3 Buscar por causa . . . 14

12 Backend: endpoints REST **14**
12.1 `GET /subscriptions/:userId` . . . 14
12.2 `POST /subscriptions` . . . 15
12.3 `DELETE /subscriptions/:endpoint` . . . 15

13 Backend: servicio de subscriptions **15**

14 Backend: mantener sincronizadas las subscriptions **16**
14.1 Solicitudes . . . 16
14.2 Comunidades . . . 16
14.3 Causas . . . 16
14.4 Acciones . . . 17

15 Backend: eventos y estrategias **17**
15.1 Publicador GraphQL + WebPush . . . 17

16 Backend: WebPushService **18**

17 Estrategias de notificacion **18**
17.1 RequestNotificationStrategy . . . 18
17.2 CauseNotificationStrategy . . . 19
17.3 ActionNotificationStrategy . . . 19

18 Service worker: recibir y pintar notificaciones **20**

19 Service worker: click en la notificacion **20**

20 Variables de entorno necesarias **21**
20.1 Backend . . . 21
20.2 Frontend . . . 21

21 Flujo completo de una notificacion de accion **21**

22 Problemas encontrados y lecciones **22**
22.1 `property endpoint should not exist` . . . 22
22.2 `relation "subscriptions" does not exist` . . . 22
22.3 `TTL header must be provided with a non-negative value` . . . 22
22.4 `causeId=undefined` en acciones . . . 22
22.5 `Strategy resolved 0 subscriptions` . . . 23

23 Checklist para probar la PWA **23**
23.1 Instalacion . . . 23

23.2 Service worker 23
23.3 Cache 23
23.4 Web push frontend 23
23.5 Web push backend 24

24 Mapa de archivos **24**
24.1 Frontend 24
24.2 Backend 24

25 Orden recomendado para explicarselo a otra persona **25**

26 Anexo: commits de la rama `next-pwa` **25**

27 Cierre **26**

1 Objetivo del documento

Este documento explica la PWA de SolidarianID desde cero, pensando en alguien que todavia no conoce bien que es una PWA ni como se conectan sus piezas.

La explicacion se ha preparado mirando la rama `next-pwa` y los commits hechos sobre `origin/develop`. El objetivo no es solo decir “que archivos hay”, sino entender el camino completo:

- como se hizo instalable el frontend;
- como se configuro Serwist y el service worker;
- que se cachea y que se fuerza a red;
- como se piden permisos de notificaciones;
- como se guarda una push subscription en backend;
- como el backend decide a quien notificar;
- como llega la notificacion al service worker;
- como se abre la URL correcta al pulsar la notificacion;
- que errores aparecieron y como se corrigieron.

2 PWA desde cero

Una PWA, o Progressive Web App, es una aplicacion web que gana capacidades normalmente asociadas a una aplicacion instalada.

En SolidarianID esas capacidades son tres:

1. **Instalacion**: el navegador puede instalar la app gracias al `manifest`, iconos y metadatos.
2. **Service worker**: un script especial vive fuera de la pagina y puede interceptar peticiones, cachear recursos y recibir eventos `push`.
3. **Web Push**: el backend puede enviar una notificacion al navegador aunque la pagina no este abierta, siempre que el usuario haya dado permiso y exista una push subscription valida.

La parte importante es que una PWA no es una tecnologia unica. Es una combinacion de piezas:

```
Manifest + iconos
        |
        v
```

```
App instalable

Service worker + cache
        |
        v
Carga mas robusta y control de red/cache

Push subscription + backend + VAPID + showNotification
        |
        v
Notificaciones del sistema
```

3 Resumen de la rama `next-pwa`

La rama se construyo de forma bastante natural. Primero se preparo la app para ser instalable, despues se metio Serwist, luego se conecto la UI de preferencias y finalmente se implemento todo el backend de web push.

3.1 Fase 1: base instalable

Commits relacionados:

- `266d455 inicio pwa`
- `c158f7b iconos para PWA`
- `4b63355 actualizacion de dependencias`
- `fdf855d fix: actualizacion configuraciones`
- `ae2ba1a fix: config`

Aqui se crearon las piezas minimas de una PWA:

- `solidarian-frontend/app/manifest.ts`
- iconos en `solidarian-frontend/public/`
- metadatos PWA en `solidarian-frontend/app/layout.tsx`
- dependencias de Serwist en `solidarian-frontend/package.json`

3.2 Fase 2: service worker y cache

Commits relacionados:

- `f27bdfd feat: service worker init`
- `1c19e27 fix: preferences notifications`
- `7050c26 fix: layout SerwistProvider`
- `15babe2 fix: sw settings`
- `99eeef9 fix: next config fixed`

Aqui se introdujo Serwist:

- `solidarian-frontend/next.config.mjs`
- `solidarian-frontend/app/serwist.ts`
- `solidarian-frontend/app/serwist/[path]/route.ts`
- `solidarian-frontend/app/sw.ts`

- `SerwistProvider` en `app/layout.tsx`

El service worker final no solo cachea. Tambien:

- evita cachear rutas sensibles;
- recibe eventos push;
- pinta notificaciones con `showNotification`;
- abre o enfoca la pagina correcta al hacer click.

3.3 Fase 3: permisos y boton de notificaciones

Commits relacionados:

- `465c193 feat: hooks pwa`
- `673d3ad fix: preferences button with control`
- `66c6b5d fix: preferencias establecidas a falso`
- `b3c41d3 fix: user precerences`

Aqui se conecto la experiencia de usuario:

- hook `solidarian-frontend/hooks/pwa/use-pwa.ts`;
- boton de preferencias en `components/preferences/`;
- valor por defecto `notificationsEnabled: false`;
- llamada a `POST /api/subscriptions` al activar notificaciones;
- llamada a `DELETE /api/subscriptions/:endpoint` al desactivarlas.

La idea de producto es buena: las notificaciones no nacen activadas por defecto. El usuario tiene que decidir activarlas.

3.4 Fase 4: backend de push subscriptions

Commits relacionados:

- `7d8694a feat: subscription repository`
- `60439f2 feat: subscription service`
- `62c1910 feat: subscription controller`
- `599894d fix: controller body`
- `9790a06 feat: migraciones de subscriptions`
- `ce00509 fix: subscription delete endpoint securizado`

Aqui se creo el almacenamiento de las push subscriptions:

- entidad de dominio `Subscription`;
- entidad TypeORM `SubscriptionEntity`;
- repositorio `SubscriptionTypeOrmAdapter`;
- servicio `SubscriptionsServiceImpl`;
- controlador REST `SubscriptionsController`;
- DTO `PushSubscriptionRequestDto`;
- migracion para la tabla `subscriptions`.

3.5 Fase 5: envio de webpush y estrategias

Commits relacionados:

- e9e220f feat: push service
- 4235b21 fix: patron estrategia
- 9e6ab48 feat: comando generar vapid keys
- cdd5adb fix: notification service arreglado definiendo tipos
- 0d96abb feat: tomar en cuenta subscripciones en cualquier momento
- 880336b feat: arreglada las sincronizaciones en requests y usadas en strategy
- 8211118 fix: tipados

Aqui se completo el ciclo de notificaciones:

- servicio `WebPushService`;
- estrategias `RequestNotificationStrategy`, `CauseNotificationStrategy`, `ActionNotificationStrategy`;
- eventos tipados en `NotificationEvent`;
- publicador `GraphqlSubscriptionsPublisherAdapter`;
- sincronizacion de los IDs seguidos por el usuario.

4 Frontend: hacer la app instalable

4.1 Manifest

El archivo principal es:

`solidarian-frontend/app/manifest.ts`

Este archivo devuelve un `MetadataRoute.Manifest` de Next:

```ts
export default function manifest(): MetadataRoute.Manifest {
  return {
    name: APP_NAME,
    short_name: APP_NAME,
    description: APP_DESCRIPTION,
    start_url: '/',
    display: 'standalone',
    background_color: '#ffffff',
    theme_color: '#000000',
    prefer_related_applications: false,
    icons: [
      {
        src: '/web-app-manifest-192x192.png',
        sizes: '192x192',
        type: 'image/png',
        purpose: 'maskable',
      },
      {
        src: '/web-app-manifest-512x512.png',
        sizes: '512x512',
        type: 'image/png',
        purpose: 'maskable',
      },
```

```
    ],
  };
}
```

Campos importantes:

- `name`: nombre completo de la app.
- `short_name`: nombre corto para launcher o pantalla de inicio.
- `start_url`: URL que abre la app instalada.
- `display: 'standalone'`: hace que se abra como app, sin barra normal del navegador.
- `theme_color`: color de la barra o marco del sistema.
- `icons`: iconos que usa el sistema al instalar.
- `purpose: 'maskable'`: permite que el icono se adapte a mascaras de Android y otros sistemas.

4.2 Metadatos en layout

En `solidarian-frontend/app/layout.tsx` se completan los metadatos:

```
export const metadata: Metadata = {
  applicationName: APP_NAME,
  title: {
    default: APP_DEFAULT_TITLE,
    template: APP_TITLE_TEMPLATE,
  },
  description: APP_DESCRIPTION,
  appleWebApp: {
    capable: true,
    statusBarStyle: 'default',
    title: APP_DEFAULT_TITLE,
  },
};
```

Esto ayuda especialmente en iOS, donde la instalacion y el modo app dependen bastante de los metadatos `appleWebApp`.

5 Frontend: Serwist y service worker

5.1 Por que Serwist

Serwist es la pieza que facilita generar y servir el service worker en una app Next moderna. En esta rama se usa con Turbopack:

```
import { withSerwist } from '@serwist/turbopack';

export default withSerwist(nextConfig);
```

Archivo:

`solidarian-frontend/next.config.mjs`

Ese wrapper conecta la configuracion de Next con Serwist.

5.2 Provider de Serwist

En `solidarian-frontend/app/serwist.ts` se exporta:

```
'use client';

export { SerwistProvider } from '@serwist/turbopack/react';
```

Y en `app/layout.tsx` se registra asi:

```
<SerwistProvider swUrl="/serwist/sw.js">{children}</SerwistProvider>
```

Esto indica al navegador donde esta el service worker generado.

5.3 Ruta que sirve el service worker

Archivo:

`solidarian-frontend/app/serwist/[path]/route.ts`

La ruta usa `createSerwistRoute`:

```
export const { dynamic, dynamicParams, revalidate, generateStaticParams, GET } = create
  additionalPrecacheEntries: [
    { url: '/~offline', revision },
    { url: '/solidarian-logo.png', revision },
    { url: '/fondo-login.jpeg', revision },
    { url: '/fondo-register.jpeg', revision },
  ],
  swSrc: 'app/sw.ts',
  useNativeEsbuild: true,
});
```

Puntos importantes:

- `swSrc: 'app/sw.ts'` dice cual es el service worker fuente.
- `additionalPrecacheEntries` mete recursos extra en precache.
- `revision` se calcula con el hash de git para invalidar cache cuando cambia la app.

6 Estrategia de cache

Archivo:

`solidarian-frontend/app/sw.ts`

La estrategia final es conservadora. No intenta cachear datos sensibles o dinamicos.

6.1 Regla 1: no cachear metodos que no sean GET

```
{
  matcher({ request }) {
    return request.method !== 'GET';
  },
```

```
  handler: new NetworkOnly(),
}
```

Esto evita cachear:

- POST;
- PUT;
- PATCH;
- DELETE.

Es importante porque esas peticiones cambian estado. Cachearlas podria provocar duplicados, acciones fantasma o respuestas antiguas.

6.2 Regla 2: no cachear rutas sensibles

```
function isSensitiveSameOriginGetRequest(request: Request, pathname: string): boolean {
  return (
    pathname.startsWith('/api/') ||
    pathname === '/graphql' ||
    pathname.startsWith('/_next/data/') ||
    pathname.includes('/requests/create') ||
    pathname === '/login' ||
    pathname === '/register' ||
    pathname.startsWith('/register/') ||
    request.headers.get('RSC') === '1'
  );
}
```

Estas rutas se tratan con `NetworkOnly`.

La razon es sencilla: en SolidarianID hay datos de usuario, autenticacion, solicitudes y contenido que cambia. Para estas rutas es mejor ir a red y fallar honestamente que enseñar datos viejos.

6.3 Regla 3: usar cache por defecto para assets seguros

Despues de las reglas sensibles entra:

```
...defaultCache
```

La cache por defecto de Serwist suele ser adecuada para recursos estaticos:

- JS y CSS generados;
- fuentes;
- imagenes;
- assets versionados.

6.4 Regla 4: limpiar caches antiguas

```
const LEGACY_RUNTIME_CACHES = ['network-first'];

self.addEventListener('activate', event => {
  event.waitUntil(Promise.all(LEGACY_RUNTIME_CACHES.map(cacheName => caches.delete(cach
});
```

Esto evita que una estrategia vieja siga viva despues de cambiar el service worker.

6.5 Regla 5: navigation preload

```
navigationPreload: true
```

`navigationPreload` permite que el navegador empiece una peticion de navegacion en paralelo mientras arranca el service worker. En la practica reduce esperas en algunas navegaciones.

7 Frontend: activacion de web push

Archivo:

`solidarian-frontend/hooks/pwa/use-pwa.ts`

Este hook concentra la logica PWA del cliente.

7.1 Comprobar soporte

```
const isSupported = useMemo(() => {
  return (
    typeof globalThis !== 'undefined' &&
    typeof globalThis.navigator !== 'undefined' &&
    'serviceWorker' in globalThis.navigator &&
    'PushManager' in globalThis &&
    'Notification' in globalThis
  );
}, []);
```

Esto evita romper en navegadores sin soporte o durante renderizado server-side.

7.2 Pedir permiso

```
const result = await globalThis.Notification.requestPermission();
setPermission(result);
```

Estados posibles:

- `default`: el usuario aun no ha decidido.
- `granted`: el usuario permite notificaciones.
- `denied`: el usuario las ha bloqueado.

Solo con `granted` se debe intentar suscribir al usuario.

7.3 Crear push subscription

La funcion `subscribeToPush` vive en:

`solidarian-frontend/features/pwa/utils.ts`

```
const sub = await registration.pushManager.subscribe({
  userVisibleOnly: true,
```

```
  applicationServerKey: urlBase64ToUint8Array(vapidPublicKey),
});
```

Puntos importantes:

- `userVisibleOnly: true` obliga a que cada push visible muestre una notificacion.
- `applicationServerKey` es la clave publica VAPID.
- `NEXT_PUBLIC_VAPID_PUBLIC_KEY` debe existir en el frontend.

7.4 Enviar la subscription al backend

El navegador genera una `PushSubscription` con:

- `endpoint`;
- clave `p256dh`;
- clave `auth`.

El hook las manda al backend:

```
await fetch('/api/subscriptions', {
  method: 'POST',
  credentials: 'include',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    endpoint: subscription.endpoint,
    p256dh,
    auth,
  }),
});
```

`credentials: 'include'` es clave porque el backend usa autenticacion por cookie/JWT de sesion.

7.5 Desactivar notificaciones

Al desactivar:

```
await unregisterSubscriptionInBackend(currentSubscription.endpoint);
await unsubscribeFromPush(currentSubscription);
```

Primero se borra en backend y despues se cancela en el navegador.

Esto evita que el backend siga intentando enviar a un endpoint que el usuario ya no quiere usar.

8 Frontend: boton de preferencias

Archivo:

`solidarian-frontend/components/preferences/preferences.tsx`

La rama corrigio un detalle importante: las notificaciones arrancan en `false`.

```
notificationsEnabled: currentUser?.preferences.notificationsEnabled ?? false
```

Al guardar preferencias:

```
if (preferences.notificationsEnabled !== initialNotificationsEnabled) {
  if (preferences.notificationsEnabled) {
    const permission = pwa.permission === 'granted'
      ? pwa.permission
      : await pwa.requestNotificationPermission();

    if (permission !== 'granted') {
      throw new Error('Debes permitir las notificaciones para activarlas.');
    }

    await pwa.subscribe();
  } else {
    await pwa.unsubscribe();
  }
}
```

La idea es:

1. El usuario activa el interruptor.
2. Se pide permiso al navegador.
3. Si acepta, se crea la subscription.
4. Se guarda en backend.
5. Se guardan preferencias.

9 Backend: modelo de subscription

Archivo:

`solidarian-id/apps/communities/src/domain/model/entities/subscription.ts`

Una subscription no es solo el endpoint del navegador. En SolidarianID tambien guarda que cosas debe recibir ese usuario:

- `userId`;
- `endpoint`;
- `p256dh`;
- `auth`;
- `requestIds`;
- `followedCommunityIds`;
- `followedCauseIds`.

La decision mas importante fue guardar listas de IDs dentro de la subscription. Asi, cuando se crea una causa o accion, el backend puede buscar rapidamente que push subscriptions estan interesadas.

```
subscriptions
  userId
  endpoint
  p256dh
  auth
  requestIds[]
```

```
  communityFollowedIds[]
  causeFollowedIds[]
```

9.1 Metodos de dominio

La entidad tiene metodos como:

- `followPendingRequest(requestId);`
- `unfollowPendingRequest(requestId);`
- `followCommunity(communityId);`
- `unfollowCommunity(communityId);`
- `followCause(causeId);`
- `unfollowCause(causeId).`

Esto evita tocar arrays “a mano” desde los servicios. La regla de no duplicar IDs vive dentro del dominio.

10 Backend: persistencia

Archivo:

`solidarian-id/apps/communities/src/infrastructure/persistence/entities/subscription.ent`

La tabla se llama `subscriptions`.

```ts
@Entity('subscriptions')
export class SubscriptionEntity {
  @PrimaryColumn('uuid')
  id: string;

  @Column()
  userId: string;

  @Column()
  endpoint: string;

  @Column()
  p256dh: string;

  @Column()
  auth: string;

  @Column('uuid', { array: true, default: [] })
  requestIds: string[];

  @Column('uuid', { array: true, default: [] })
  communityFollowedIds: string[];

  @Column('uuid', { array: true, default: [] })
  causeFollowedIds: string[];
}
```

La migracion crea esta tabla:

`solidarian-id/apps/communities/src/infrastructure/persistence/migrations/1777232397594-Auto.ts`

11 Backend: consultas importantes

Archivo:

`solidarian-id/apps/communities/src/infrastructure/persistence/repositories/subscription.typeorm.repository.ts`

11.1 Buscar por usuario

Se usa para notificar cambios de estado de solicitudes:

```
findByUserId(userId)
```

11.2 Buscar por comunidad

Se usa cuando se crea una causa dentro de una comunidad:

```
.where(':communityId = ANY(subscription.communityFollowedIds)', { communityId })
```

Si un usuario sigue esa comunidad y tiene push subscription, entra en el envio.

11.3 Buscar por causa

Se usa cuando se crea una accion dentro de una causa:

```
.where(':causeId = ANY(subscription.causeFollowedIds)', { causeId })
```

Si un usuario sigue esa causa y tiene push subscription, entra en el envio.

12 Backend: endpoints REST

Archivo:

`solidarian-id/apps/communities/src/infrastructure/presentation/controllers/rest/subscri`

12.1 `GET /subscriptions/:userId`

Devuelve el resumen de lo que sigue un usuario:

- solicitudes pendientes;
- comunidades seguidas;
- causas seguidas;
- eventos;
- objetivos.

Importante: este endpoint no significa que exista una push subscription del navegador. Es un resumen funcional de follows.

12.2 POST /subscriptions

Registra la push subscription del usuario autenticado.

Body:

```
{
  "endpoint": "https://fcm.googleapis.com/fcm/send/...",
  "p256dh": "...",
  "auth": "..."
}
```

El DTO valida los campos:

```
@IsString()
@IsNotEmpty()
endpoint: string;
```

Esto fue necesario porque el backend usa `ValidationPipe` con whitelist. Sin decoradores, Nest rechazaba propiedades validas con errores como:

```
property endpoint should not exist
```

12.3 DELETE /subscriptions/:endpoint

Elimina la push subscription del usuario autenticado. El endpoint se manda con `encodeURIComponent` desde frontend porque contiene caracteres como `/`, `:` y otros simbolos.

13 Backend: servicio de subscriptions

Archivo:

```
solidarian-id/apps/communities/src/application/services/subscriptions.service.impl.ts
```

Cuando se registra una subscription, el backend no guarda una fila vacia. Primero mira el estado actual del usuario:

```
const followedIdsOrError = await this.getUserSubscriptions(userId, {
  requestStatus: RequestStatus.PENDING,
  includeRequestIds: true,
  includeFollowedCauseIds: true,
  includeFollowedCommunityIds: true,
});
```

Despues crea la subscription con esos IDs:

```
Subscription.create({
  userId,
  endpoint,
  p256dh,
  auth,
  requestIds: followedIdsOrError.value.requestIds,
  followedCommunityIds: followedIdsOrError.value.followedCommunityIds,
```

```
  followedCauseIds: followedIdsOrError.value.followedCauseIds,
});
```

Esto fue una correccion clave. Si el usuario ya seguia comunidades o causas antes de activar notificaciones, la subscription debe nacer sincronizada.

14 Backend: mantener sincronizadas las subscriptions

Una vez guardada la subscription, el usuario puede seguir o dejar de seguir cosas. Por eso la rama anadio sincronizacion en varios flujos.

14.1 Solicitudes

Archivo:

`solidarian-id/apps/communities/src/application/services/requests.service.impl.ts`

Cuando se crea una solicitud de comunidad, causa o accion, se mete el `requestId` en las subscriptions del usuario:

```
syncUserRequestSubscriptions(userId, requestId)
```

Cuando la solicitud cambia de estado, se publica:

```
publishRequestStatusChanged(...)
```

Asi el usuario puede recibir “solicitud aprobada” o “solicitud rechazada”.

14.2 Comunidades

Archivo:

`solidarian-id/apps/communities/src/application/services/communities.service.impl.ts`

Cuando un usuario entra en una comunidad:

```
syncUserCommunitySubscriptions(userId, communityId, 'add')
```

Cuando sale:

```
syncUserCommunitySubscriptions(userId, communityId, 'remove')
```

Cuando se crea una comunidad con causas iniciales, tambien se sincronizan esas causas para el creador y se publica `CAUSE_CREATED` para cada causa inicial.

14.3 Causas

Archivo:

`solidarian-id/apps/communities/src/application/services/causes.service.impl.ts`

Cuando un usuario sigue una causa:

```
syncUserCauseSubscriptions(userId, causeId, 'add')
```

Cuando deja de seguirla:

```
syncUserCauseSubscriptions(userId, causeId, 'remove')
```

Cuando se crea una causa, se publica:

```
publishCauseCreated(cause)
```

14.4 Acciones

Archivo:

`solidarian-id/apps/communities/src/application/services/actions.service.impl.ts`

Cuando se crea una accion:

```
await this.subscriptionsPublisher.publishActionCreated(action);
```

Esto dispara el flujo de webpush para quienes sigan la causa de esa accion.

15 Backend: eventos y estrategias

Archivo:

`solidarian-id/apps/communities/src/domain/services/notification.service.ts`

La rama termino definiendo eventos planos:

```
export interface CauseNotificationEvent {
  eventType: 'CAUSE';
  id: string;
  communityId: string;
}

export interface ActionNotificationEvent {
  eventType: 'ACTION';
  id: string;
  causeId: string;
}
```

Esto arreglo un problema importante: al hacer spread de entidades de dominio, los getters como `causeId` podian perderse. La solucion correcta fue construir payloads explicitos con strings.

15.1 Publicador GraphQL + WebPush

Archivo:

`solidarian-id/apps/communities/src/infrastructure/presentation/controllers/graphql/subs`
`subscriptions.publisher.ts`

Este adaptador hace dos cosas:

1. Publica el evento GraphQL para clientes conectados.
2. Llama a `webPushService.sendNotification(...)` para usuarios con push subscription.

Ejemplo con acciones:

```
await this.webPushService.sendNotification({
  eventType: 'ACTION',
  id: action.id.value,
  name: action.name,
  description: action.description,
  causeId: action.causeId,
});
```

16 Backend: WebPushService

Archivo:

`solidarian-id/apps/communities/src/application/services/web-push.service.impl.ts`

El servicio configura VAPID:

```
webpush.setVapidDetails(
  process.env.VAPID_SUBJECT!,
  process.env.VAPID_PUBLIC_KEY!,
  process.env.VAPID_PRIVATE_KEY!,
);
```

Despues busca una estrategia capaz de manejar el evento:

```
const strategy = this.strategies.find(s => s.canHandle(event));
```

Y envia:

```
webpush.sendNotification(
  {
    endpoint: subscription.endpoint,
    keys: {
      p256dh: subscription.p256dh,
      auth: subscription.auth,
    },
  },
  payloadJson,
  {
    TTL: this.ttlSeconds,
  },
);
```

El TTL es importante. Sin el header TTL, el envio puede fallar con:

```
TTL header must be provided with a non-negative value
```

17 Estrategias de notificacion

17.1 RequestNotificationStrategy

Archivo:

`solidarian-id/apps/communities/src/application/services/strategies/request-notification.strategy.ts`

Uso:

- evento `REQUEST_STATUS_CHANGED`;
- busca subscriptions por `userId`;
- payload tipo `REQUEST_STATUS_CHANGED`;
- abre `/requests` al hacer click.

```
Solicitud aprobada/rechazada
        |
        v
findByUserId(userId)
        |
        v
Enviar push al creador de la solicitud
```

17.2 CauseNotificationStrategy

Archivo:

`solidarian-id/apps/communities/src/application/services/strategies/cause-notification.strategy.ts`

Uso:

- evento `CAUSE`;
- busca subscriptions por `communityId`;
- payload tipo `CAUSE`;
- abre la causa creada.

```
Nueva causa en comunidad
        |
        v
findByCommunityId(communityId)
        |
        v
Enviar push a quienes siguen la comunidad
```

17.3 ActionNotificationStrategy

Archivo:

`solidarian-id/apps/communities/src/application/services/strategies/action-notification.strategy.ts`

Uso:

- evento `ACTION`;
- busca subscriptions por `causeId`;
- consulta la causa para obtener `communityId`;
- payload tipo `ACTION`;
- abre la accion creada.

```
Nueva accion en causa
        |
        v
findByCauseId(causeId)
        |
        v
Enviar push a quienes siguen la causa
```

18 Service worker: recibir y pintar notificaciones

Archivo:

`solidarian-frontend/app/sw.ts`

El backend envia un payload como este:

```
{
  "title": "Mira tus acciones",
  "message": "Se ha creado una nueva accion en una de tus causas",
  "type": "ACTION",
  "data": {
    "communityId": "...",
    "causeId": "...",
    "actionId": "..."
  }
}
```

El service worker lo recibe:

```
self.addEventListener('push', (event: PushEvent) => {
  if (!event.data) {
    return;
  }

  const payload = event.data.json() as PushNotificationPayload;

  event.waitUntil(self.registration.showNotification(payload.title, options));
});
```

`showNotification` no pinta nada dentro de la app. Pinta una notificacion del sistema operativo o del navegador.

Por eso, si la PWA esta abierta y no se ve nada dentro de la pagina, eso no significa necesariamente que falle. La notificacion aparece fuera de la UI normal.

19 Service worker: click en la notificacion

El handler de click:

```
self.addEventListener('notificationclick', (event: NotificationEvent) => {
  const notification = event.notification;
  const data = notification.data as StoredNotificationData | undefined;
```

```
  notification.close();

  const urlToOpen = getNotificationUrl(data);

  event.waitUntil(openOrFocusWindow(urlToOpen));
});
```

La URL se calcula en:

`solidarian-frontend/features/pwa/utils.ts`

Reglas:

- `ACTION` abre `/communities/:communityId/causes/:causeId/actions/:actionId`;
- `CAUSE` abre `/communities/:communityId/causes/:causeId`;
- `REQUEST_STATUS_CHANGED` abre `/requests`;
- si faltan datos, abre `/notifications`.

La funcion `openOrFocusWindow` intenta enfocar una ventana ya abierta antes de abrir una nueva. Eso mejora la experiencia en escritorio y PWA.

20 Variables de entorno necesarias

20.1 Backend

Variables esperadas por `WebPushService`:

```
VAPID_SUBJECT=mailto:admin@solidarianid.com
VAPID_PUBLIC_KEY=...
VAPID_PRIVATE_KEY=...
```

La rama anadio el comando:

`npm run generate:vapid:keys`

en `solidarian-id/package.json`.

20.2 Frontend

Variable esperada por `subscribeToPush`:

`NEXT_PUBLIC_VAPID_PUBLIC_KEY=...`

Debe coincidir con la clave publica del backend.

21 Flujo completo de una notificacion de accion

Este es el flujo mas completo porque cruza casi todas las piezas.

```
1. Usuario activa notificaciones en preferencias
2. Frontend pide Notification.permission
3. Frontend crea PushSubscription con PushManager
4. Frontend manda endpoint, p256dh y auth a POST /api/subscriptions
```

```
5. Backend guarda la subscription con las causas/comunidades que sigue el usuario
6. Se crea una accion dentro de una causa
7. ActionsService publica ACTION_CREATED y publishActionCreated
8. GraphqlSubscriptionsPublisherAdapter crea un payload ACTION plano
9. WebPushService elige ActionNotificationStrategy
10. ActionNotificationStrategy busca subscriptions por causeId
11. WebPushService envia con web-push y VAPID
12. El service worker recibe push
13. El service worker llama showNotification
14. El usuario pulsa la notificacion
15. notificationclick abre la ruta de la accion
```

22 Problemas encontrados y lecciones

22.1 `property endpoint should not exist`

Causa:

- el DTO no tenia decoradores de validacion;
- el `ValidationPipe` del backend tenia whitelist activa.

Solucion:

- añadir `@IsString()` y `@IsNotEmpty()` al DTO.

22.2 `relation "subscriptions" does not exist`

Causa:

- el codigo esperaba la tabla `subscriptions`;
- la base no tenia migracion aplicada.

Solucion:

- crear migracion para `subscriptions`;
- ejecutar migraciones en la base de communities.

22.3 `TTL header must be provided with a non-negative value`

Causa:

- `web-push` necesitaba un TTL explicito.

Solucion:

- enviar `{ TTL: 3600 }`.

22.4 `causeId=undefined` en acciones

Causa:

- se estaba construyendo el evento con spread sobre una entidad de dominio;
- algunos datos venian de getters y se perdian.

Solucion:

- definir eventos planos;
- construir payloads explicitos.

22.5 `Strategy resolved 0 subscriptions`

No siempre es un bug de la estrategia.

Puede significar:

- el usuario sigue la causa/comunidad, pero no activo push;
- no existe fila en `subscriptions`;
- la fila existe, pero no tiene sincronizado `causeFollowedIds` o `communityFollowedIds`;
- la subscription se creo antes de arreglar la sincronizacion.

La rama lo arreglo sincronizando:

- solicitudes al crearlas;
- comunidades al entrar/salir;
- causas al seguir/dejar de seguir;
- estado inicial al registrar una push subscription.

23 Checklist para probar la PWA

23.1 Instalacion

- Abrir la app en navegador compatible.
- Verificar que existe manifest.
- Verificar iconos 192x192 y 512x512.
- Instalar la app.
- Confirmar que abre en modo standalone.

23.2 Service worker

- Abrir DevTools.
- Ir a Application > Service Workers.
- Confirmar que `/serwist/sw.js` esta activo.
- Confirmar que no hay service worker antiguo.
- Probar una navegacion.

23.3 Cache

- Confirmar que assets estaticos se cachean.
- Confirmar que `/api/*` va a red.
- Confirmar que `/graphql` va a red.
- Confirmar que login, register y requests create no se sirven desde cache.

23.4 Web push frontend

- Confirmar `Notification.permission === 'granted'`.

- Confirmar que existe `NEXT_PUBLIC_VAPID_PUBLIC_KEY`.
- Activar notificaciones desde preferencias.
- Verificar que se llama `POST /api/subscriptions`.
- Desactivar notificaciones.
- Verificar que se llama `DELETE /api/subscriptions/:endpoint`.

23.5 Web push backend

- Confirmar `VAPID_SUBJECT`, `VAPID_PUBLIC_KEY` y `VAPID_PRIVATE_KEY`.
- Confirmar que la tabla `subscriptions` existe.
- Confirmar que existe una fila para el usuario.
- Confirmar que `causeFollowedIds` y `communityFollowedIds` tienen IDs.
- Crear causa dentro de comunidad seguida.
- Crear accion dentro de causa seguida.
- Aceptar/rechazar solicitud.

24 Mapa de archivos

24.1 Frontend

- `solidarian-frontend/app/manifest.ts`: manifest instalable.
- `solidarian-frontend/app/layout.tsx`: metadatos y `SerwistProvider`.
- `solidarian-frontend/app/serwist.ts`: export cliente del provider.
- `solidarian-frontend/app/serwist/[path]/route.ts`: ruta que sirve el service worker.
- `solidarian-frontend/app/sw.ts`: service worker, cache, push y click.
- `solidarian-frontend/hooks/pwa/use-pwa.ts`: hook de permisos y suscripcion.
- `solidarian-frontend/features/pwa/utils.ts`: utilidades de PushManager, VAPID y URLs.
- `solidarian-frontend/features/pwa/types/payload.type.ts`: tipos del payload push.
- `solidarian-frontend/components/preferences/preferences.tsx`: guardado de preferencias y activacion.
- `solidarian-frontend/components/preferences/preferences-notification.tsx`: interruptor visual.

24.2 Backend

- `solidarian-id/apps/communities/src/domain/model/entities/subscription.ts`: entidad de dominio.
- `solidarian-id/apps/communities/src/infrastructure/persistence/entities/subscriptio` entidad TypeORM.
- `solidarian-id/apps/communities/src/infrastructure/persistence/repositories/subscri typeorm.repository.ts`: consultas.
- `solidarian-id/apps/communities/src/application/services/subscriptions.service.impl` alta/baja de push subscriptions.
- `solidarian-id/apps/communities/src/infrastructure/presentation/controllers/rest/sul` endpoints REST.
- `solidarian-id/apps/communities/src/application/services/web-push.service.impl.ts`: envio con `web-push`.

- `solidarian-id/apps/communities/src/application/services/strategies/request-notification.strategy.ts`: solicitudes.
- `solidarian-id/apps/communities/src/application/services/strategies/cause-notification.strategy.ts`: causas.
- `solidarian-id/apps/communities/src/application/services/strategies/action-notification.strategy.ts`: acciones.
- `solidarian-id/apps/communities/src/infrastructure/presentation/controllers/graphql/subscriptions.publisher.ts`: puente GraphQL + webpush.
- `solidarian-id/apps/communities/src/domain/services/notification.service.ts`: tipos de eventos.
- `solidarian-id/apps/communities/src/infrastructure/persistence/migrations/177723239Auto.ts`: tabla `subscriptions`.

25 Orden recomendado para explicarselo a otra persona

Si tienes que defender o presentar esta PWA, este orden funciona bien:

1. Explicar que una PWA no es solo “instalar una web”: necesita manifest, service worker y permisos.
2. Enseñar `manifest.ts` y los iconos.
3. Enseñar `SerwistProvider` y `/serwist/sw.js`.
4. Enseñar `sw.ts`, primero cache y luego push.
5. Explicar que la UI de preferencias no envia push; solo pide permiso y registra la subscription.
6. Enseñar `POST /subscriptions`.
7. Enseñar la tabla `subscriptions`.
8. Explicar que los arrays de IDs son el indice funcional para saber a quien notificar.
9. Enseñar estrategias: request por usuario, cause por comunidad, action por causa.
10. Cerrar con el flujo completo: crear accion, encontrar usuarios, enviar push, recibir en service worker, abrir URL.

26 Anexo: commits de la rama `next-pwa`

Commits propios de la rama comparados con `origin/develop`, en orden cronologico:

```
266d455 inicio pwa
778cae1 Ignorar ficheros de certificados en el frontend y archivos de service worker er
c158f7b iconos para PWA
4b63355 actualizacion de dependencias
fdf855d fix: actualizacion configuraciones
7c06347 fix: variables entorno docker
7d8694a feat: subscription repository
60439f2 feat: subscription service
62c1910 feat: subscription controller
e9e220f feat: push service
4235b21 fix: patron estrategia
f27bdfd feat: service worker init
465c193 feat: hooks pwa
1c19e27 fix: preferences notifications
7050c26 fix: layout SerwistProvider
```

```
b3c41d3 fix: user precerences
673d3ad fix: preferences button with control
ce00509 fix: subscription delete endpoint securizado
99eeef9 fix: next config fixed
f10d515 fix: actor fallo
07f3317 fix: test arreglados
9e6ab48 feat: comando generar vapid keys
ae2ba1a fix: config
15babe2 fix: sw settings
66c6b5d fix: preferencias establecidas a falso
7617cff feat: utils metodos
a6de067 fix: subscripcion ahora registra la lista de solicitudes
599894d fix: controller body
9790a06 feat: migraciones de subscriptions
cdd5adb fix: notification service arreglado definiendo tipos
0d96abb feat: tomar en cuenta subscripciones en cualquier momento
880336b feat: arreglada las sincronizaciones en requests y usadas en strategy
8211118 fix: tipados
```

27 Cierre

La PWA de SolidarianID quedo como un flujo completo:

```
Usuario activa notificaciones
        |
        v
Frontend crea PushSubscription
        |
        v
Backend guarda endpoint + claves + IDs seguidos
        |
        v
Dominio publica eventos de request/cause/action
        |
        v
Estrategia selecciona destinatarios
        |
        v
WebPushService envia payload
        |
        v
Service worker muestra notificacion
        |
        v
Click abre la pagina correcta
```

El punto mas importante para entenderla es este: **seguir una comunidad o causa no es lo mismo que tener notificaciones activadas**. Para recibir push hacen falta las dos cosas:

- relacion funcional en SolidarianID;

- push subscription valida guardada en backend.